

What is socket?

Socket is defined as communication endpoint.

Sockets can be used for bi-directional communication.

Using socket one process can communicate with another process on same machine (UNIX socket) or different machine (INET sockets) in the network.

Sockets can also be used for communication over bluetooth, CAN, etc.

terminal> man 7 socket

What Is Socket Programming?

Think of two people talking on the phone.

One person is the server: they wait for calls.

The other is the client: they make the call.

The phone line is like the socket — it connects them.

Socket System Calls

```
fd = socket(addr_family, sock_type, protocol);
```

arg1: addr_family = AF_UNIX or AF_INET

arg2: sock_type = SOCK_STREAM or SOCK_DGRAM

arg3: protocol = 0

Socket information is maintained in struct sockaddr. It will differ for each address family.

struct sockaddr

sa_family -- address family (AF_UNIX or AF_INET)

struct sockaddr_un

sun_family -- address family = AF_UNIX

sun_path -- UNIX socket file path

struct sockaddr_in

sin_family -- address family = AF_INET

sin_port -- port number in big endian format

sin_addr -- struct in_addr -- ip address in big endian format

ret = bind(fd, &sock_addr, sock_length)

arg1: socket fd (created by socket() call)

arg2: address of sockaddr_un or sockaddr_in

arg3: sizeof(sockaddr_un) or sizeof(sockaddr_in)

returns 0 on success.

ret = connect(fd, &sock_addr, sock_length)

arg1: socket fd (created by socket() call)

arg2: address of sockaddr_un or sockaddr_in

arg3: sizeof(sockaddr_un) or sizeof(sockaddr_in)

returns 0 on success.

cli_fd = accept(fd, &sock_addr, &sock_length)

arg1: socket fd (created by socket() call)

arg2: (out-param) address of sockaddr_un or sockaddr_in

arg3: (in-out param) sizeof(sockaddr_un) or sizeof(sockaddr_in)

returns 0 on success.

listen(fd, pending);

arg1: socket fd (created by socket() call)

arg2: max pending socket connection requests

shutdown(fd, SHUT_RDWR);

arg1: socket fd (created by socket() call)

arg2: which connections to be closed -- SHUT_RDWR.

Term Meaning

IP Address Like your home address on the internet (e.g., 192.168.1.1)

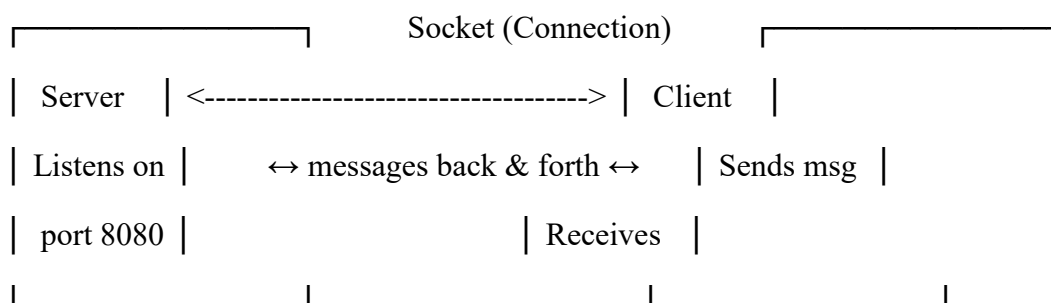
Port Like a door number at your home (e.g., 8080)

Server A program that waits for requests (e.g., a restaurant)

Client A program that sends a request (e.g., a customer)

Socket A connection pipe between client and server

Visualize It.



Now Break Down Socket Programming into 5 Steps (Just Concepts)

Step What You Do Example

- 1 Create a socket "Plug in the phone"
- 2 Bind to IP & Port "Give the phone a number"
- 3 Listen "Wait for a call"
- 4 Accept "Pick up the phone when someone calls"
- 5 Communicate "Talk: Send/Receive message"

Server Side:

1. socket()
2. bind()
3. listen()
4. accept()
5. recv() / send()
6. close()

Client Side:

1. socket()
2. connect()
3. send() / recv()
4. close()

C Functions:

Function	What It Does
socket()	Creates a socket
bind()	Assigns IP address & port to the socket
listen()	Waits for clients
accept()	Accepts client connection
recv()	Receives message from client
send()	Sends message to client
close()	Closes the socket

Step 1: socket() — Creating the Socket

What does it do?

It creates a virtual pipe to connect two devices.

socket() Syntax in C

```
int socket(int domain, int type, int protocol);
```

parameters:

Parameter	Meaning
-----------	---------

domain	Address family — use AF_INET for IPv4
--------	---------------------------------------

type	Communication type — use SOCK_STREAM for TCP
------	--

protocol	Usually 0 (let system choose default)
----------	---------------------------------------

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

sockfd is your socket file descriptor

If it returns -1, that means socket creation failed

Basic C Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <sys/socket.h>
```

```
#include <netinet/in.h> // For AF_INET
```

```
#include <unistd.h> // For close()
```

```
int main()
```

```
{
```

```

int sockfd;

// Create socket
sockfd = socket(AF_INET, SOCK_STREAM, 0);

if (sockfd < 0) {
    perror("Socket creation failed");
    exit(1);
}

printf("Socket created successfully! sockfd = %d\n", sockfd);

// Close the socket
close(sockfd);

return 0;
}

```

output:

Socket created successfully! sockfd = 3

Create socket `socket(AF_INET, SOCK_STREAM, 0)`

Check success if return `>= 0`, it's successful

Socket FD Used in next steps (like `bind()`, `connect()`, etc.)

Header File Why It is Included

stdio.h For input/output functions like printf() and perror()

stdlib.h For general utilities like exit()

sys/socket.h Contains functions and constants for creating and managing sockets (socket(), AF_INET, SOCK_STREAM)

netinet/in.h Defines Internet address family (struct sockaddr_in, INADDR_ANY, htons())

unistd.h Provides access to the close() function to close the socket

```
int sockfd;
```

Declares an integer variable sockfd to store the socket file descriptor.

It is like a file handle used to identify the socket for later operations.

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

socket() Function Breakdown:

```
int socket(int domain, int type, int protocol);
```

Parameter	Value Used	Meaning
domain	AF_INET	Use IPv4 Internet protocol
type	SOCK_STREAM	TCP connection (reliable, connection-oriented)
protocol	0	Default protocol for TCP

```
close(sockfd);
```

Closes the socket file descriptor.

Frees up the resources associated with it.

